

Name: Ashritha Hegde

SRN No.: PES1UG24CS089

Semester: V

Section: B

Course Code: UE24CS341A

Course Name: Software Engineering

LAB 3 – Architectural Analysis and Component Design

Link: <https://github.com/ashrithahegde10/SE-LABS-PES1UG24CS089>

1. Comparison of Architectural Styles

Before selecting an architecture for the **Alumni Mentorship & Mock Interview Platform**, three common architectural styles were considered: Layered Architecture, Microservices Architecture, and Client-Server Architecture.

Architecture	Structure	Advantages	Disadvantages	Suitability
Layered Architecture	Presentation → Business → Data	Simple to understand, well organized, and easy to maintain	Components can become tightly connected, and scaling individual parts can be difficult	Best
Microservices Architecture	Independent services such as Mentor Matching, Session Management, and Feedback	Services can be developed, deployed, and scaled independently	More complex to develop and maintain, and communication between services can cause network delays	Good
Client-Server Architecture	Client communicates with a central server	Simple structure, centralized control, and easier data management	The central server can become a bottleneck or single point of failure	Moderate

Selected Architecture

After comparing the three options, Layered Architecture was selected for this system.

The main reason is that the system has clearly separated functions such as user interaction, mentor recommendation, session booking and availability management, feedback and scorecard management, and data storage. These responsibilities can be organized into separate layers, making the system easier to understand, maintain, and develop.

2. Architecture Selection – Layered Architecture

Why Layered Architecture?

The Alumni Mentorship & Mock Interview Platform has different responsibilities that can be separated into presentation, business logic, and data management. This makes Layered Architecture suitable for the system.

Reason 1 – Clear Separation of Responsibilities

The User Interface handles interaction with students, alumni mentors, and administrators, while components such as Mentor Matching and Session Management handle the main application logic.

The Database is responsible for storing user, mentor, session, availability, scorecard, and feedback information. This separation makes the system easier to develop and maintain.

Reason 2 – Easier Maintenance

Changes to one part of the system can be made without significantly affecting unrelated parts.

For example, the mentor recommendation logic can be modified without changing the User Interface or the database structure. Similarly, changes to feedback and scorecard handling can be made independently from session booking.

Security Advantage

Layered Architecture prevents the User Interface from directly accessing the database.

Business components can perform authentication, authorization, input validation, and access control before information is stored or retrieved. This helps protect sensitive information such as user details, mentor information, bookings, and interview feedback.

Performance Advantage

The Mentor Matching and Session Management modules can be optimized within the business layer to support the required response time for mentor recommendations and availability information.

This helps the system meet the requirement that mentor recommendations and availability should be provided within 3 seconds under normal conditions.

3. Component Identification

The following five components are identified for the system.

1. User Interface

This component provides the interface through which students, alumni mentors, and administrators interact with the platform. It handles:

- Mentor recommendation requests
- Viewing mentor availability
- Booking mentorship and mock interview sessions
- Viewing interview feedback
- User account-related interactions

2. Session Management

This component manages mentorship and mock interview sessions. It handles:

- Viewing mentor availability
- Session booking
- Session scheduling
- Session-related information
- Managing session details

3. Mentor Matching

This component is responsible for recommending suitable mentors to students. It handles:

- Mentor recommendation requests
- Matching mentors based on domain or relevant requirements
- Providing suitable mentor information

4. Feedback & Scorecard

This component manages mock interview scorecards and feedback. It handles:

- Structured interview scorecards
- Mentor feedback
- Submission of interview feedback
- Providing feedback information to students

5. Database

This component stores and manages the data required by the platform. It handles:

- User information
- Mentor information
- Mentor availability
- Session and booking information
- Scorecards and interview feedback

4. Provided and Required Interfaces

The main interfaces are:

Component	Provided Interface	Required Interface
User Interface	—	Mentor Recommendation Request, Session Booking / Availability
Mentor Matching	Mentor Recommendation Request	—
Session Management	Session Booking / Availability	Session & Availability Data
Feedback & Scorecard	—	Scorecard & Feedback Data
Database	Session & Availability Data, Scorecard & Feedback Data	—

In implementation, these component interfaces can be realized through appropriate APIs or service calls between the application layers.

5. Recommended Component Connections

Connection 1 - User Interface → Mentor Matching

Interface: Mentor Recommendation Request

Purpose:

Used to request suitable mentor recommendations based on the student's required domain or mentoring needs.

Connection 2 - User Interface → Session Management

Interface: Session Booking / Availability

Purpose:

Used to view mentor availability and book mentorship or mock interview sessions.

Connection 3 - Session Management → Database

Interface: Session & Availability Data

Purpose:

Used to store and retrieve mentor availability, session schedules, and booking information.

Connection 4 - Feedback & Scorecard → Database

Interface: Scorecard & Feedback Data

Purpose:

Used to store and retrieve structured mock interview scorecards and feedback information.

6. UML Component Diagram

